

CON10X Web Domain

Web Transformation Engine

Architecture Design v1.1

OPEN SOURCE: LICENSED UNDER AGPL v3.0

© 2026 Russel Hawkins. Licensed under the GNU Affero General Public License v3.0 (AGPL-3.0).

1. The ConteX Law

The ConteX Law: "Structure + Behaviour + Influence + Objective fed to AI eliminates hallucination. In any domain. Every time."

The ConteX Law is the foundational principle governing every component of this architecture. Ambiguity in AI input is the sole cause of hallucination in AI output. By eliminating ambiguity through four non-overlapping, deterministic pillars of structured context, the AI model receives a complete and unambiguous specification. The result is deterministic, production-ready output. Every time.

This is not a theory. It is a proven engineering principle implemented in production by SnapStak, the web transformation platform that applies ConteX to the specific domain of website deconstruction and framework code generation.

1.1 The Four Pillars

Pillar 1: Structure — What is it

Structure is the immutable geometric and hierarchical truth of the component. Captured directly from the live browser using `getBoundingClientRect()` and `getComputedStyle()` — not inferred from source code. Serialised as an SVG file containing exact pixel positions, dimensions, colour values, text content, and containment hierarchy. The SVG is always the source of truth. The AI transcribes — it does not interpret.

```
Storage: {STORAGE_ROOT}/{userUUID}/{componentId}/{componentId}.svg
```

Pillar 2: Behaviour — What does it do

Raw CSS computed properties are captured as `{componentId}_css.json` and raw JavaScript interactions as `{componentId}_js.json`. These JSON files are processed asynchronously by the OpenRouter AI Gateway, which produces semantic markdown descriptions: `{componentId}_css.md` and `{componentId}_js.md`. These describe meaning — not raw code.

```
Storage: {STORAGE_ROOT}/{userUUID}/{componentId}/{componentId}_css.md + _js.md
```

Pillar 3: Influence — What shaped it

Browser name and version, operating system, screen resolution, device pixel ratio, viewport dimensions, user agent, and media feature queries. All captured at the moment of DOM serialisation. Stored as `{componentId}_influence.json` on disk.

```
Storage: {STORAGE_ROOT}/{userUUID}/{componentId}/{componentId}_influence.json
```

Pillar 4: Objective — What must it achieve

The only pillar not derived from the DOM. User-defined at conversion time: target device type, screen width, and target framework. Without Objective the AI guesses the target. With Objective the AI transcribes to a precise specification. Captured in two phases: inferred at extraction time from live viewport width, and confirmed by the user at conversion time.

```
Storage: {STORAGE_ROOT}/{userUUID}/{componentId}/{componentId}_objective.json
```

1.2 Why This Eliminates Hallucination

Every other AI coding system begins with ambiguous input: a prompt, a description, a raw HTML dump. The AI fills gaps with training data — which is guessing. ConteX eliminates the gaps before the AI is invoked. The four pillars form a complete, non-overlapping specification. There are no gaps to fill. The AI transcribes — it does not interpret. Identical input produces identical output. Every time. This is deterministic AI.

1.3 The Kitchen Analogy

Structure is the ingredients. Behaviour is the recipe (browser-computed truth). Influence and Objective are the cooking utensils. Without all four: you can produce something, it will taste wrong, and it will taste differently wrong every time. Competitors hand the AI only the ingredients. ConteX provides all four. Non-deterministic output is the industry's dirty secret. ConteX makes it deterministic.

1.4 Ephemeral Pillar Storage

All four pillar files are written to the customer's local drive per authenticated use and deleted immediately after successful code production by the Constructor.

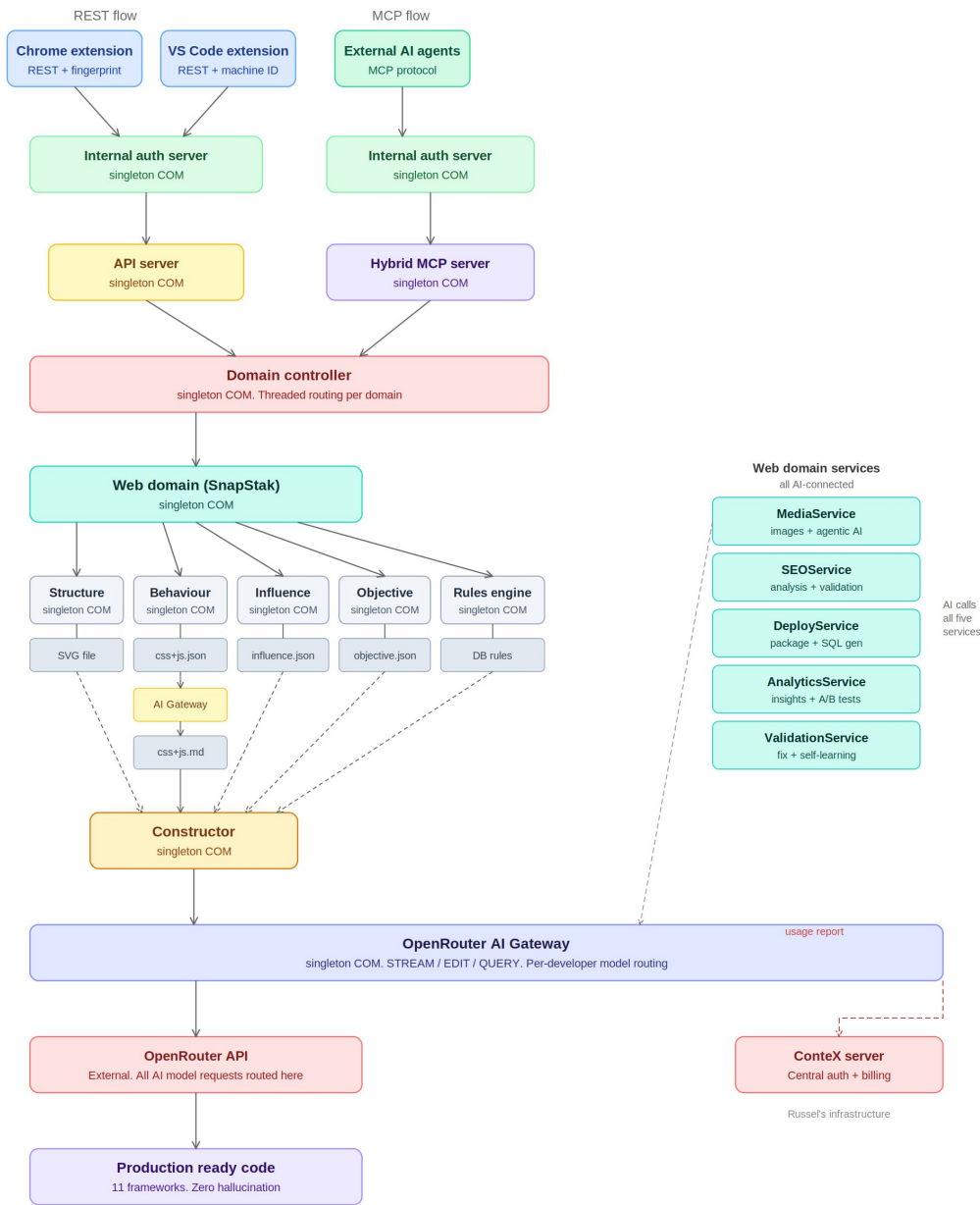
- Zero persistent data exposure on customer infrastructure.
- No server-side state accumulation between requests.
- Compliance with data minimisation: data exists only as long as necessary to produce the output.

NOTE: If code production fails, pillar files are retained for retry. Cleanup is triggered only on confirmed successful delivery.

2. System Architecture Overview

ConteX is architected as a white-label, on-premise .NET 8 platform. The entire processing stack runs on the customer's own infrastructure. All components from authentication through to the Constructor are implemented as singleton COM objects with threaded methods. All AI operations across the entire platform route exclusively through the OpenRouter AI Gateway singleton — no component calls OpenRouter directly.

2.1 Architecture Diagram



ConteX Symbiotic Agentic Context Engine — Architecture v1.1
© 2026 Russel Hawkins. All rights reserved. CONFIDENTIAL.

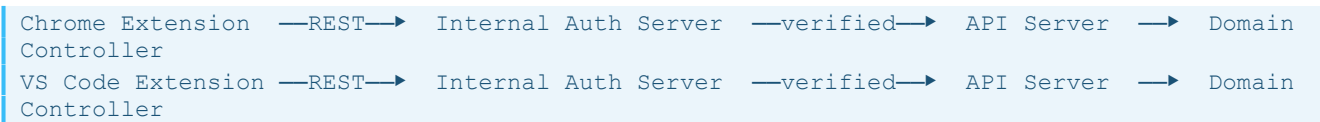
Figure 1: CON10X Web Domain — Complete Architecture v1.1

2.2 Architecture Principles

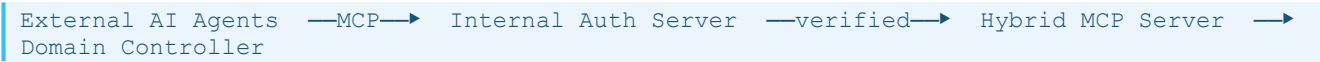
- All processing components are .NET 8 singleton COM objects (net8.0-windows TFM, registered via regasm).
- Each singleton spawns a new thread per request — concurrent processing with zero instantiation overhead.
- No customer data leaves the customer's infrastructure during processing.
- The ConteX algorithm is compiled to a binary COM object — source code is never exposed.
- All AI requests across the entire platform route through the OpenRouter AI Gateway singleton without exception.
- The Web Domain singleton orchestrates all five web services: MediaService, SEOService, DeployService, AnalyticsService, and ValidationService.
- Usage reporting is the only outbound connection — from the OpenRouter AI Gateway directly to the centralised ConteX billing server at contex.ai/billing.
- All pillar files are ephemeral — written per request, deleted on successful completion.

2.3 Two Authenticated Entry Flows

REST Flow: Chrome Extension and VS Code Extension



MCP Flow: External AI Agents



2.4 Component Registry

Component	Type	Responsibility
Internal Auth Server	Singleton COM	Validates all inbound requests. Four-layer security model.
API Server	Singleton COM	REST gateway for Chrome and VS Code extensions.
Hybrid MCP Server	Singleton COM	MCP protocol interface for external AI agents.
Domain Controller	Singleton COM	Routes authenticated requests to registered domain singletons.
Web Domain (SnapStak)	Singleton COM	Orchestrates pillar agents and all five web services.
Structure Agent	Singleton COM	Receives DOM snapshot. Produces SVG file.
Behaviour Agent	Singleton COM	Receives CSS/JS JSON. Calls AI Gateway. Produces .md files.
Influence Agent	Singleton COM	Receives browser environment data. Produces influence.json.

Objective Agent	Singleton COM	Receives user target selection. Produces objective.json.
Rules Engine	Singleton COM	Queries domain rules DB. Manages contex_rules table.
Constructor	Singleton COM	Assembles all pillar outputs. Calls AI Gateway. Cleans up files.
MediaService	Web Domain Service	Asset management. AI image generation, editing, analysis, diagrams.
SEOService	Web Domain Service	SEO analysis, navigation validation, placeholder detection.
DeployService	Web Domain Service	Project packaging, scaffold assembly, schema formatting, SQL script generation.
AnalyticsService	Web Domain Service	User behaviour, A/B testing, AI insights and anomaly detection.
ValidationService	Web Domain Service	Code validation, AI fixing. Self-learning rule discovery.
OpenRouter AI Gateway	Singleton COM	Single point of contact for all AI operations. STREAM/EDIT/QUERY.

3. Four-Layer Security Model

ConteX implements a four-layer security model. All four layers must be satisfied before any request reaches the processing pipeline. The Internal Auth Server validates each layer sequentially. Failure at any layer results in immediate rejection.

3.1 Layer 1: ConteX Developer API Key

Every API request must carry a valid developer API key in the Authorization header. Format: {API_KEY_PREFIX}{48-character-string}, where API_KEY_PREFIX is defined in contex.config and is unique per white-label customer deployment.

```
Authorization: Bearer {API_KEY_PREFIX}{48-character-random-string}
```

NOTE: The API_KEY_PREFIX (e.g. "sk-acme-") is set once per white-label deployment in contex.config. Never "sk-or-v1-" (OpenRouter) or any other reserved prefix.

3.2 Layer 2: User UUID

The authenticated user UUID is validated against three identity tables in the ConteX database: google_users (Google OAuth), github_users (GitHub OAuth), and users (email registration). The lookup cascades through all three to locate the identity and retrieve the associated active AI model for routing by the AI Gateway.

3.3 Layer 3: Browser Fingerprint (Chrome Extension Only)

The Chrome extension captures a stable browser fingerprint at DOM serialisation time using only invariant data points to prevent false rejections.

Stable Fingerprint Components

Data Point	Rationale
browserName	Chrome — stable, only changes on complete reinstall.
browserMajor	Major version only (e.g. "124") — stable for months. Minor version excluded.
platform	"Win32", "MacIntel", "Linux x86_64" — stable for the life of the machine.
devicePixelRatio	1 or 2 — stable for a given monitor. Excluded when external monitors detected.

The fingerprint is SHA-256 hashed on the extension side before transmission. On first use from a new machine, the hash is recorded in contex_browser_sessions. Version tolerance: if the browserMajor advances by exactly 1, the hash is accepted and the stored record is updated. Seat enforcement prevents use on more machines than the allocated seat count.

NOTE: The browser fingerprint layer applies only to the REST flow from the Chrome extension. The MCP flow and VS Code extension flow do not use a browser fingerprint.

3.4 Layer 4: VS Code Machine ID (VS Code Extension Only)

The VS Code extension provides a machine-specific identifier validated by the Auth Server. Machine IDs are registered at first use in contex_vscode_machines and linked to the developer's API key. The maximum number of VS Code registrations per developer key is controlled by contex_developer_keys.max_vscode_seats.

3.5 Developer API Key Hierarchy

Level	Description
-------	-------------

Level 1: ConteX Master OpenRouter Key	Held exclusively by Russel Hawkins. The provisioning key used to create all customer sub-keys via OpenRouter's provisioning API.
Level 2: Customer OpenRouter Sub-Key	Provisioned per white-label customer. One sub-key per customer account. Never exposed to the customer or their developers.
Level 3: Developer OpenRouter Sub-Key	Provisioned per developer. One per developer. Linked to the customer sub-key. Never exposed to the developer. Enables per-developer usage tracking at the OpenRouter level.
Level 4: Developer ConteX API Key	Format: {API_KEY_PREFIX}{48 chars}. This is the only key the developer ever sees. Stored as SHA-256 hash only — shown once at provisioning, never stored in plaintext.

3.6 Centralised ConteX Billing Server

OpenRouter AI Gateway —HTTPS usage report—> context.ai/billing (Russel's infrastructure)

NOTE: The usage report payload: { customerId, developerId, modelId, tokensConsumed, operationType, timestamp }. No component data or source code transmitted.

4. ConteX Database Schema

ConteX uses a dedicated MySQL database (context) that is completely independent of any other application database. ConteX owns and manages this database entirely. There is no dependency on SnapStak or any other system database.

4.1 google_users / github_users / users

Column	Purpose
id	Auto-increment internal primary key. Never exposed to clients.
uid	UUID v4. Public identifier. Used in all API calls and JWT tokens.
email	User email address. Unique.
name	Display name from OAuth provider.
active_model_id	Currently selected AI model ID routed through OpenRouter.
passive_model_id	Secondary model for Behaviour pillar async descriptions.
plan_type	Subscription tier: free, pro, ultra, teams.
openrouter_api_key	Developer's OpenRouter sub-key. Stored AES-256-GCM encrypted.

4.2 context_developer_keys

Column	Purpose
user_id	UUID of the developer.
context_api_key_hash	SHA-256 hash of the developer's ConteX API key. Shown once at creation, never stored in plaintext.
context_api_key_prefix	First 12 characters for display purposes (e.g. "sk-acme-4J00").
openrouter_api_key_enc	Developer's OpenRouter sub-key, AES-256-GCM encrypted. Decrypted only at request time.
openrouter_key_hash	SHA-256 hash of the OpenRouter sub-key for reference.
is_active	1 = active, 0 = disabled.
max_browser_seats	Maximum number of Chrome extension registrations for this developer key.
max_vscode_seats	Maximum number of VS Code extension registrations for this developer key.
daily_request_limit	Maximum API requests per day. NULL = unlimited.
monthly_cost_limit	Maximum monthly spend in USD. NULL = unlimited.
requests_today	Rolling daily counter. Reset by scheduled task at midnight.
is_rate_limited	1 = rate limited. Set automatically on limit breach.
created_at	Key creation timestamp.

expires_at	Key expiry. NULL = never expires.
------------	-----------------------------------

4.3 contex_browser_sessions

Column	Purpose
id	Auto-increment primary key.
developer_key_user_id	UUID linking to contex_developer_keys.
fingerprint_hash	SHA-256 hash of the stable browser fingerprint.
browser_name	e.g. Chrome. Stored for version tolerance logic.
browser_major	Major version integer. Used for the +1 tolerance check.
platform	e.g. Win32, MacIntel.
machine_label	Optional human-readable label (e.g. "John's Work Laptop").
registered_at	First registration timestamp.
last_seen_at	Last successful authentication timestamp.
is_active	1 = active. Set to 0 to revoke a specific machine.

4.4 contex_vscode_machines

Column	Purpose
id	Auto-increment primary key.
developer_key_user_id	UUID linking to contex_developer_keys.
machine_id	SHA-256 hashed VS Code machine identifier.
machine_label	Optional human-readable label.
registered_at	First registration timestamp.
last_seen_at	Last successful authentication timestamp.
is_active	1 = active.

4.5 contex_rules

Column	Purpose
rule_id	Auto-increment primary key.
domain_id	Domain: e.g. web, healthcare, finance, legal.
rule_category	Category: e.g. layout, typography, interaction, html-to-jsx.
trigger_context	JSON object. Conditions under which this rule applies.
rule_content	The rule in structured natural language.

priority	Application priority. Higher = applied first.
is_active	Enable/disable without deletion.
validated_by	Human validator who approved this rule.

NOTE: The `context_tag_query` self-learning table from v1.0 has been removed. Unknown entity discovery is now handled within the SVG Structure pillar pipeline. The Rules Engine table grows through human-validated rule promotion only.

4.6 context_api_usage_logs

Column	Purpose
id	Auto-increment primary key.
user_id	Developer UUID.
model_id	AI model used for this request.
operation_type	QUERY, EDIT, or STREAM.
input_tokens	Tokens consumed in the prompt.
output_tokens	Tokens consumed in the response.
cost	USD cost of this request.
status	success or error.
response_time_ms	End-to-end latency in milliseconds.
created_at	Timestamp of the request.

5. .NET 8 Singleton COM Object Design

All ConteX processing components are implemented as .NET 8 singleton COM objects targeting the net8.0-windows TFM, registered with the Windows COM subsystem via regasm. Each request spawns a new thread from the singleton, enabling concurrent processing without instantiation overhead.

5.1 IAuthServer — Internal Authentication

```
public interface IAuthServer : IConteXComponent
{
    string ValidateRequest(string contxApiKey, string userUUID,
                          string browserFingerprint, string vsMachineId);
    string ValidateMCPRequest(string contxApiKey, string userUUID);
    string ProvisionDeveloper(string licenceKey, string developerEmail,
                             int maxBrowserSeats, int maxVsCodeSeats);
}
```

5.2 IOpenRouterAIGateway — AI Gateway

```
public interface IOpenRouterAIGateway : IConteXComponent
{
    string Query(string authContext, string prompt, string modelId);
    string Edit(string authContext, string prompt, string existingCode, string modelId);
    void Stream(string authContext, string prompt, string modelId,
               Action<string> onChunk, Action onComplete, Action<string> onError);
    void ReportUsage(string customerId, string developerId, string modelId,
                    int inputTokens, int outputTokens, string operationType);
}
```

5.3 Singleton COM Registration Pattern

```
[ComVisible(true), ClassInterface(ClassInterfaceType.None)]
[Guid("B8C9D0E1-F2A3-4567-1234-678901234567")]
[ProgId("ConteX.OpenRouterAIGateway")]
public class OpenRouterAIGatewayCOM : IOpenRouterAIGateway
{
    private static readonly OpenRouterAIGatewayCOM _instance = new();
    private OpenRouterAIGatewayCOM() { }
    public static OpenRouterAIGatewayCOM Instance => _instance;
}
```

6. OpenRouter AI Gateway

The OpenRouter AI Gateway is the single point of contact for all AI operations across the entire ConteX platform. No component calls OpenRouter directly.

6.1 Operation Types

Operation	Used By	Operation Type	Model
QUERY	Behaviour Agent, SEOService, AnalyticsService, MediaService (analysis)	Single-turn. Temperature 0.1.	developer active_model_id
EDIT	ValidationService (code fixing), DeployService (SQL scripts)	Receives existing content plus instruction. Returns modified output only.	developer active_model_id
STREAM	Constructor (code gen), MediaService (image gen/edit)	Streams token-by-token via SSE. 30-second keepalive pings.	bytedance- seed/seedream-4.5 for images. active_model_id for code.

6.2 Retry and Resilience

```
maxRetries:      3
baseDelayMs:     1000 / maxDelayMs: 10000 / backoffMultiplier: 2
retryableStatusCodes: [429, 500, 502, 503, 504]
requestTimeout:  3600000ms (1 hour)
keepaliveInterval: 30000ms (SSE ping)
temperature:     0.1 (deterministic output)
```

7. REST API Specification

7.1 Authentication Headers

```
Authorization: Bearer {API_KEY_PREFIX}{48-character-key}
X-User-UUID:      {user UUID}
X-Browser-FP:     {SHA-256 browser fingerprint hash}    // Chrome extension only
X-VSCode-Machine: {SHA-256 VS Code machine ID hash}     // VS Code extension only
Content-Type:     application/json
```

7.2 Core Endpoints

POST /api/web-to-svg/transform

Initiates the extraction pipeline. Synchronously returns the Structure SVG. Behaviour AI processing runs asynchronously after response is sent.

```
Request: { domSnapshot, componentId, componentCSS, componentJS, influence, objective }
Response: { success, svg, objectCount, width, height, sourceUrl }
```

POST /api/svg-to-code/generate

Stage 2. Loads all four pillar files, assembles the ConteX blueprint, invokes the AI Gateway STREAM operation.

```
Request: { componentId, framework, language }
Response: { success, componentCode, componentCSS }
```

POST /api/svg-to-code/convert

Stage 3. Converts Stage 2 output to a specific framework. Can be called multiple times without re-running Stage 2.

```
Request: { componentId, html, css, framework, language }
```

7.3 Supported Frameworks

Framework ID	Framework	File Extension
react	React	.jsx / .tsx
nextjs	Next.js	.jsx / .tsx
vue	Vue 3	.vue
nuxt	Nuxt 3	.vue
svelte	Svelte	.svelte
angular	Angular	.ts
solidjs	SolidJS	.jsx / .tsx
alpine	Alpine.js	.html
vanilla	Vanilla JS	.js
tailwind	Tailwind CSS	.html / .jsx
astro	Astro	.astro

8. MCP Protocol Specification

The Hybrid MCP server implements the Model Context Protocol standard. External AI agents connect using standard MCP tooling and are routed internally to the Domain Controller COM object. The MCP flow validates only Layer 1 (ConteX API key) and Layer 2 (user UUID).

8.1 MCP Server Configuration

```
{
  "mcpServers": { "contex": { "command": "contex-mcp-server",
    "env": {
      "CONTEX_API_KEY": "{API_KEY_PREFIX}{48-char-key}",
      "CONTEX_USER_UUID": "{userUUID}",
      "CONTEX_ENDPOINT": "https://{customer-domain}/mcp"
    }
  }
}
```

8.2 MCP Tool Definitions

```
contex_extract: { required: ["componentId", "domSnapshot"] }
contex_generate: { required: ["componentId", "framework"] }
contex_status: { required: ["componentId"] }
```

9. SnapStak Web Domain: Extraction and Generation Pipelines

9.1 Extraction Pipeline

- Chrome extension serialises the live DOM using `getBoundingClientRect()` and `getComputedStyle()`. Direct browser truth — no Puppeteer.
- Extension sends serialised DOM, CSS, JS, browser environment, and user objective to API server via REST.
- Internal Auth Server validates all four security layers.
- API Server routes to Domain Controller → Web Domain → Structure Agent.
- Structure Agent synchronously builds the SVG tree. Returns SVG to client immediately.
- Behaviour Agent fires asynchronously. Sends CSS and JS JSON to AI Gateway QUERY. Writes `_css.md` and `_js.md`.
- Influence Agent writes `{componentId}_influence.json` to disk immediately after SVG.
- Objective Agent writes `{componentId}_objective.json` to disk immediately after SVG.

9.2 Code Generation Pipeline

- VS Code extension sends `componentId`, framework, and language to API server.
- Internal Auth Server validates ConteX API key, user UUID, and VS Code machine ID.
- Constructor loads all four pillar files from the component directory.
- Constructor assembles the complete ConteX blueprint.
- Constructor sends blueprint to AI Gateway STREAM operation via developer's OpenRouter sub-key.
- AI Gateway streams production-ready code back via SSE.
- AI Gateway reports token usage to centralised ConteX billing server.
- Constructor deletes all four pillar files from disk on success.
- ValidationService validates the generated code. Calls AI Gateway EDIT if fixes are needed.
- DeployService assembles the complete project package including framework scaffold, database schema, and AI-generated SQL scripts.
- Complete packaged project delivered to VS Code extension as a downloadable ZIP.

9.3 Storage Layout

```
{STORAGE_ROOT}/{userUUID}/{componentId}/
├── {componentId}.svg           # Structure – synchronous write
├── {componentId}_css.md       # Behaviour CSS – async AI Gateway QUERY
├── {componentId}_js.md        # Behaviour JS – async AI Gateway QUERY
├── {componentId}_influence.json # Influence – synchronous write
├── {componentId}_objective.json # Objective – synchronous write
└── [all files deleted after successful generation]
```

NOTE: Hidden component sub-directories may exist under `{componentId}/components/` for accordion and overflow-hidden elements captured during the two-pass reflow expansion.

10. Deployment Architecture

10.1 White-Label Deployment

- Contex.dll — compiled binary containing all singleton COM objects including the AI Gateway.
- Contex.MCP.exe — the Hybrid MCP server process.
- Contex.API.exe — the REST API server process.
- MySQL database (contex) — authentication, developer key management, usage tracking, and rules storage.
- Component storage volume — local disk path for ephemeral pillar files.
- Contex licence key — validates the installation against the centralised Contex billing server at startup.

10.2 Environment Variables

Variable	Purpose
CONTEX_DB_CONNECTION	MySQL connection string for the Contex database.
CONTEX_STORAGE_PATH	Local filesystem path for ephemeral pillar files.
CONTEX_BILLING_ENDPOINT	URL of centralised Contex billing server (contex.ai/billing).
CONTEX_LICENCE_KEY	Installation licence key. Validated at startup.
CONTEX_OPENROUTER_ENDPOINT	OpenRouter API base URL.
CONTEX_ENCRYPTION_KEY	AES-256-GCM encryption key. 256-bit, base64-encoded. Never logged.
CONTEX_ENCRYPTION_IV_SALT	Salt used to derive per-record IVs. Prevents IV reuse across records.
API_KEY_PREFIX	Customer-specific API key prefix (e.g. "sk-acme-"). Set once per deployment.
CONTEX_BEHAVIOUR_MODEL	Fallback AI model ID for Behaviour pillar QUERY operations.
CONTEX_GENERATOR_MODEL	Fallback AI model ID for Constructor STREAM operations.
CONTEX_MAX_TOKENS	Maximum tokens per AI request. Default: 32000.
CONTEX_TEMPERATURE	AI temperature. Default: 0.1 for deterministic output.

10.3 Outbound Network Requirements

Destination	Protocol	Purpose
openrouter.ai	HTTPS/443	All AI model invocations via the AI Gateway singleton. Routed through developer sub-keys.
contex.ai/billing	HTTPS/443	Usage reporting from AI Gateway to centralised billing server.
contex.ai/auth	HTTPS/443	Licence key validation at startup.

NOTE: No component data, no source code, no pillar content is transmitted to any external endpoint. Only usage metrics and licence validation are sent externally.

11. Encryption Specification

OpenRouter API keys are sensitive credentials that must be protected at rest. Contex uses AES-256-GCM authenticated encryption for all stored OpenRouter keys.

11.1 Algorithm

- Algorithm: AES-256-GCM (authenticated encryption with associated data).
- Key size: 256 bits. Stored in CONTEX_ENCRYPTION_KEY environment variable.
- IV: 96-bit random IV generated per record. Stored alongside the ciphertext.
- Authentication tag: 128-bit GCM tag. Verifies integrity on decryption.
- Storage format: base64(IV) + "." + base64(ciphertext + tag).

11.2 Key Lifecycle

- Encrypted at creation time when the developer key is provisioned.
- Decrypted in memory only at request time, immediately before the OpenRouter API call.
- Plaintext key is never written to disk, never logged, and never transmitted outside the request thread.
- The developer Contex API key is shown once at provisioning time. Only the SHA-256 hash is stored.

11.3 Key Rotation

If CONTEX_ENCRYPTION_KEY must be rotated, the process is: decrypt all records with the old key, re-encrypt with the new key, update CONTEX_ENCRYPTION_KEY. A Contex admin utility will be provided. Key rotation does not require issuing new developer API keys.

12. Intellectual Property and License

Copyright Notice

© 2026 Russel Hawkins. Licensed under the GNU Affero General Public License v3.0 (AGPL-3.0).

CON10X Web Domain, the ConteX Law, the Four Pillars (Structure, Behaviour, Influence, Objective), the ConteX Rules Engine architecture, the OpenRouter AI Gateway design, the five web domain service architecture, the four-layer security model with browser fingerprint registration, the developer key hierarchy, and all architectural designs, interface definitions, data models, and implementation specifications contained in this document are released as open source under the GNU Affero General Public License v3.0 (AGPL-3.0). Copyright © 2026 Russel Hawkins.

License

This document and the CON10X Web Domain source code are released under the GNU Affero General Public License v3.0 (AGPL-3.0). You are free to use, study, modify, and redistribute this work under the terms of that license. Source is published at github.com/SnapStak-AI/SnapStak-Web-Domain.

Source Availability

The CON10X Web Domain algorithm, its implementation, the four-pillar context assembly pipeline, the browser fingerprint stability model, and the developer key encryption scheme are published as open source under AGPL-3.0. The complete source is available at github.com/SnapStak-AI/SnapStak-Web-Domain, and contributions and derivative works are welcome under the terms of that license.